

Zeitplanung

Programmierprojekt (DarkIT)

Memory (Data Security)

Ein edukatives Memory-Spiel mit Datensicherheitsthema

Student: Mathis Fingerhut

Matrikelnummer: 1816794

Team: Einzelprojekt

Studiengang: Angewandte Informatik

Fakultät: Fakultät IV - Wirtschaft und Informatik

Hochschule: Hochschule Hannover

Hannover, 30.10.2025

Inhaltsverzeichnis

1. Projektübersicht

- 1.1 Projektziele
- 1.2 Projektumfang
- 1.3 Technologie-Stack
- 1.4 Zeitrahmen

2. Schritte zur Erstellung des Zeitplans

- 2.1 Projektziele definieren
- 2.2 Aufgaben identifizieren
- 2.3 Aufgaben priorisieren
- 2.4 Ressourcen zuweisen
- 2.5 Zeitaufwand schätzen
- 2.6 Meilensteine festlegen

3. Detaillierter Zeitplan

4. Benötigte Ressourcen

- 4.1 Software-Ressourcen
- 4.2 Fachliche Ressourcen
- 4.3 Materialien

5. Risikomanagement

- 5.1 Risikoanalyse
- 5.2 Puffer und Eskalation

6. Meilensteine und Erfolgskriterien

- 6.1 Haupt-Meilensteine
- 6.2 Erfolgskriterien

7. Überwachung und Berichtsmethoden

8. Fazit

1 Projektübersicht

1.1 Projektziele

Memory (Data Security) ist ein Java-basiertes Memory-Spiel, das Spielspaß mit Lerninhalten zur Datensicherheit verbindet. Ziel ist ein flüssiges Gameplay, intuitive Bedienung, edukative Inhalte und stabile Builds (JAR-Release).

1.2 Projektumfang

- **Gameplay:** Karten-Matching mit Zeitlimit (vllt Zuglimit)
- **Kartensets:** 1 Themengebiet (Datensicherheit)
- **UI/UX:** Hauptmenü, Spielscreen, Highscore-Anzeige
- **Audio:** Sound-Effekte für Kartenaktionen

1.3 Technologie-Stack

- **Programmiersprache:** Java 17+
- **GUI-Framework:** Swing
- **Build/Run:** Maven/Gradle, JAR Packaging
- **Versionskontrolle:** Git/GitHub
- **UML/Docs:** PlantUML, Javadoc

1.4 Zeitrahmen

12 Wochen (ca. 3 Monate)

2 Schritte zur Erstellung des Zeitplans

2.1 Projektziele definieren

Ziele für Gameplay, Lerninhalte, UI/UX und Performance wurden festgelegt.

2.2 Aufgaben identifizieren

Aufgaben in Phasen gegliedert: Initiierung, Analyse, Design, Implementierung (3 Phasen), Testing/Optimierung, Dokumentation, Abschluss.

2.3 Aufgaben priorisieren

Design vor Implementierung; Kern-Matching-Logik vor Features; Testing parallel; Dokumentation kontinuierlich.

2.4 Ressourcen zuweisen

Einzelentwickler mit folgenden Rollen: Gameplay, UI/UX, Content, Testing.

2.5 Zeitaufwand schätzen

Eine Woche pro Phase; Pufferzeit pro Woche ca. 10-15%.

2.6 Meilensteine festlegen

Meilensteine decken wöchentliche Ziele und Abgabetermine ab.

3 Detaillierter Zeitplan

Woche	Aktivität	Aufgaben	Verantwortlich	Meilenstein
1	Projektinitiierung	• Kick-off, Ziele definieren • Technologie-Stack bestätigen • Repository einrichten	Mathis Fingerhut	Projektstart
2	Anforderungsanalyse	• Use-Cases definieren • Spielregeln festlegen • GUI-Entwürfe skizzieren	Mathis Fingerhut	Anforderungen klar
3	Anwendungsfälle definieren	• Use-Case-Diagramme • Spielablauf-Szenarien • UI-Navigation-Flows	Mathis Fingerhut	Use Cases fertig
4	Klassenmodell entwerfen	• Klassendiagramm (Card, GameBoard, GameLogic, ScoreManager, AudioManager)	Mathis Fingerhut	Klassenmodell erstellt
5	Architekturentwurf	• Package-Struktur • Game-Loop Design • Asset-Management Konzept	Mathis Fingerhut	Architektur definiert
6	Implementierung Phase 1	• Card-Klasse, GameBoard Grundgerüst • Basic Game-Loop • Input-Handling	Mathis Fingerhut	Grundgerüst steht
7	Implementierung Phase 2	• Matching-Logik implementieren • Score-System • Timer und Zug-Zählung	Mathis Fingerhut	Kern-Logik funktioniert
8	Implementierung Phase 3	• Highscore-System • Audio-Integration • Karten-Animationen	Mathis Fingerhut	Features komplett
9	UI/UX Polish	• Visuelle Verbesserungen • Responsive Design • Menü-Screens finalisieren	Mathis Fingerhut	UI finalisiert

Woche	Aktivität	Aufgaben	Verantwortlich	Meilenstein
10	Testing & Bugfixing	• Unit-Tests für Matching-Logik • UI-Testing • Performance-Checks	Mathis Fingerhut	Spiel stabil
11	Dokumentation	• Javadoc, README • UML finalisieren • Nutzerdokumentation	Mathis Fingerhut	Dokumentation fertig
12	Projektabschluss	• JAR erstellen • Finaler Test • Abgabe vorbereiten	Mathis Fingerhut	Projekt abgegeben

4 Benötigte Ressourcen

4.1 Software-Ressourcen

IntelliJ IDEA, Java 17+, Git/GitHub, PlantUML, Image Editing Tools, Audio Software

4.2 Fachliche Ressourcen

Java/Swing, OOP, 2D-Grafik-Programmierung, Event-Handling, Datei-I/O

4.3 Materialien

- Karten-Designs (PNG/SVG)
- Datensicherheits-Begriffe und Erklärungen
- Audio-Dateien (Sound-Effekte)
- Icons für UI-Elemente

5 Risikomanagement

5.1 Risikoanalyse

Risiko	Auswirkung	Maßnahme
Matching-Logik fehlerhaft	Spiel unspielbar	Umfangreiches Testing, einfache Algorithmen zuerst
Performance bei vielen Karten	Ruckeln, schlechte UX	Lazy Loading, effiziente Datenstrukturen
Zeitmangel für UI-Polish	Schlechte Benutzererfahrung	MVP-Ansatz, Kernfunktionen priorisieren
Assets nicht konsistent	Unprofessionelles Erscheinungsbild	Style-Guide erstellen, einheitliche Größen
JAR-Packaging Probleme	App nicht lauffähig	Frühzeitig testen, Maven/Gradle nutzen

5.2 Puffer und Eskalation

Wöchentliche Kontrolle; bei Problemen sofort auf MVP reduzieren und Stabilität priorisieren.

6 Meilensteine und Erfolgskriterien

6.1 Haupt-Meilensteine

- **Woche 4:** Design-Dokumente abgeschlossen
- **Woche 7:** Kern-Matching-Logik implementiert
- **Woche 9:** Alle Features integriert
- **Woche 11:** Testing abgeschlossen
- **Woche 12:** Projekt abgegeben

6.2 Erfolgskriterien

- Spiel mit 1 Karten-Set funktionsfähig
- Fehlerfreie Matching-Logik
- Flüssiges Gameplay
- Lauffähige JAR-Datei mit allen Assets
- Vollständige Dokumentation
- Termingerechte Abgabe

7 Überwachung und Berichtsmethoden

Wöchentliche Selbstreflexion, regelmäßige Git Commits, Issue-Tracking mit GitHub; Anpassungen bei technischen Problemen; MVP-Fokus bei Verzögerungen.

8 Fazit

Dieser Zeitplan bietet eine klare Struktur für die Entwicklung von Memory (Data Security). Mit Fokus auf Kernfunktionen, regelmäßigem Testing und kontinuierlicher Dokumentation ist eine termingerechte Abgabe eines qualitativ hochwertigen Spiels realistisch.